Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Partie II

Les structures de données

1. Préambule
2. Les Listes
3. Les ensembles
4. Les tuples
5. Les dictionnaires
6. La copie de structures

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Préambule

Les **structures de données** sont des moyens spécifiques d'**organiser et de stocker des données** afin qu'elles puissent être **consultées et travaillées de manière efficace.** Les structures de données constituent la **relation entre les données et les opérations** pouvant être effectuées dessus.

Le **choix d'une structure** de données dépend du **problème** que nous résolvons et du **type de données** dont nous disposons. Une structure de données **inappropriée** peut entraîner des **durées** de fonctionnement (inutilement) longues, des **pertes de mémoire** et un gaspillage de stockage.

Il faut donc bien choisir sa structure de donnée en fonction de son besoin, il en existe plusieurs et elles sont imbriquables.

Voici les principales structures de données en python :

- Liste : `list()` / `[]`
- Ensemble : `set()`
- Dictionnaire : `dict()` / `{}`
- Tuple : `tuple()` / `(,)`

Structures	ordonnée	modifiable	indexé	éléments uniques
Liste	✓	✓	✓	✗
Ensemble	✗	✓ / ✗*	✗	✓
Dictionnaire	✓	✓ / ✗**	✓(clé)	✓ / ✗**
Tuples	✓	✗	✓	✗

* Les éléments d'un ensemble ne sont pas modifiables "en place", mais on peut ajouter et supprimer des éléments d'un ensemble

** Les *clés* d'un dictionnaire sont uniques et non modifiables, mais les *valeurs* (éléments référencés par ces clés) sont modifiables et les doublons sont possibles.

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Définition

Une liste est une **collection ordonnée d'objets** arbitraires. En d'autres termes, c'est une **structure de données**, pourvue d'un **index**, permettant de **stocker** toutes sortes d'objets : nombres, chaînes et même d'autres listes.

Propriétés des listes :

- Elles définissent un **ordre de positionnement** de leurs éléments, accessibles par un indice (ou index)
- Elles sont de **longueur variable, hétérogènes et mixables**
- Elles sont **modifiables "sur place"** : on peut y modifier, y ajouter ou y supprimer des éléments sans devoir créer une nouvelle liste

Déclarer et initialiser une liste

On peut déclarer une liste vide :

Syntaxe (Déclaration d'une liste vide)

```
liste = []
liste = list()
```

✎ Pratique pour stocker les résultats que l'on va obtenir dans la suite d'un programme !

Ou avec des valeurs directement :

Syntaxe (Déclaration d'une liste avec des éléments)

```
liste = [1, 2, 3, 4]
liste = ["girafe", "tigre", "singe", "souris"]
liste = [1, "girafe", "a", 3.14]
```

✎ Astuce : par défaut, une liste vide est `False`, alors que si elle contient quelque chose, elle est `True`.

Les indices d'une liste

On peut accéder aux éléments d'une liste par leur **position**. Cette position est régie par un **numéro** qui est appelé **indice** (ou index) de la liste.

Une liste et ses indices

liste : ["girafe", "tigre", "singe", "souris"]
indices : 0 1 2 3

Remarques :

- Les indices d'une liste de n éléments commencent à 0 et se terminent à $n-1$
- Pour récupérer la taille d'une liste, on utilise `len(liste)` .

Accéder aux éléments d'une liste (indice)

On peut accéder à un élément d'une liste grâce à son indice :

Syntaxe (Accès à un élément de la liste)

```
liste = [1, 2, 3, 4]
liste[1] #récupère '2', car l'indice commence à 0
```

Attention à ne pas dépasser la taille de la liste !

Code (Erreur out of range)

```
1 liste[4]
2 Traceback (innermost last):
3 File "<stdin>", line 1, in ?
4 IndexError: list index out of range
```

Accéder aux éléments d'une liste

On peut aussi récupérer plusieurs éléments, grâce aux tranches (*slicing* en anglais)

Syntaxe (Accéder à une tranche de la liste)

```
liste[debut : fin : pas]
```

On récupère les objets de la liste présents entre le début et la fin - 1, de pas en pas.

✎ Fonctionne de la même manière que la fonction `range` dans le `for`

Accéder aux éléments d'une liste (indice)

Quelques exemples de slicing :

Syntaxe (Accès à une tranche de la liste)

```
liste = [1, 2, 3, 4]
liste[1:3] #récupère 2 et 3
liste[1:] #tout à partir de l'indice 1 : 2, 3, 4
liste[:3] #tout jusqu'à l'indice 3 : 1, 2, 3
```

- Attention à l'inclusion et l'exclusion, l'intervalle de la tranche est systématiquement de ce type : [début:fin[
  Le début est inclus et la fin exclue.

Accéder aux éléments d'une liste (indice)

On peut aussi accéder aux éléments de la liste en partant de la fin, **le dernier élément est à l'indice -1**, le précédant en -2 etc.

Syntaxe (Accès à un élément de la liste en partant de la fin)

```
liste = [1, 2, 3, 4]
liste[-1] # récupère 4
liste[-2] # récupère 3
```

Pour récapituler !

Une liste et ses indices

liste	: ["girafe",	"tigre",	"singe",	"souris"]
indices en partant du début :	0	1	2	3
indices en partant de la fin :	-4	-3	-2	-1

Liste 2D - Tableaux - Matrices

En python, on peut représenter les tableaux et les matrices comme des listes de listes.

Code (Liste 2D)

```
1 liste2D = [[1,2,3],[4,5,6],[7,8,9]]
2
3 # ou de cette façon pour plus de clareté
4
5 liste2D = [[1,2,3],
6            [4,5,6],
7            [7,8,9]]
```

1	2	3
4	5	6
7	8	9

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Liste 2D - Opérations

Toutes les opérations sur les listes 1D fonctionnent de la même manière sur les listes 2D. Il faut d'abords accéder au premier index, puis au deuxième.

Code (Liste 2D)

```
1 liste2D = [[27,63,33,85],
2            [16,29,99,6],
3            [8,47,0,56],
4            [76,41,25,30]]
```

	0	1	2	3
0	27	63	33	85
1	16	29	99	6
2	8	47	0	56
3	76	41	25	30

Liste 2D - Opérations

On peut récupérer une ligne entière avec son indice.

Code (Liste 2D)

```
1 liste2D = [[27,63,33,85],
2            [16,29,99,6],
3            [8,47,0,56],
4            [76,41,25,30]]
5
6 ligne2 = liste2D[1]
7 print(ligne2) #affiche [16,29,99,6]
```

	0	1	2	3
0	27	63	33	85
1	16	29	99	6
2	8	47	0	56
3	76	41	25	30

Liste 2D - Opérations

On peut récupérer une cellule en particulier grâce à ses deux indices :

Code (Liste 2D)

```
1 liste2D = [[27,63,33,85],
2            [16,29,99,6],
3            [8,47,0,56],
4            [76,41,25,30]]
5
6 ligne2 = liste2D[2][3]
7 print(ligne2) #affiche 56
```

	0	1	2	3
0	27	63	33	85
1	16	29	99	6
2	8	47	0	56
3	76	41	25	30

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Opérations sur les listes

Soit une liste :

```
1 ma_liste = [1, 2, 3 ,4, 5]
```

- Accès par indice :

```
1 print(ma_liste[0])  # Output: 1
2 print(ma_liste[-1]) # Output: 5 (dernier élément)
```

- Modifier un élément :

```
1 ma_liste[1] = 10
2 print(ma_liste)  # Output: [1, 10, 3, 4, 5]
```

- Ajouter à la fin : `append()`

```
1 ma_liste.append(6)
2 print(ma_liste)  # Output: [1, 10, 3, 4, 5, 6]
```

- Ajouter à une position spécifique : `insert()`

```
1 ma_liste.insert(2, 15)
2 print(ma_liste)  # Output: [1, 10, 15, 3, 4, 5, 6]
```

Opérations sur les listes

- Supprimer par valeur : `remove()`

```
1 ma_liste.remove(10)
2 print(ma_liste)  # Output: [1, 15, 3, 4, 5, 6]
```

- Supprimer par indice : `del` ou `pop()`

```
1 del ma_liste[1]
2 print(ma_liste)  # Output: [1, 3, 4, 5, 6]
3
4 valeur = ma_liste.pop(2)
5 print(ma_liste)  # Output: [1, 3, 5, 6]
6 print(valeur)  # Output: 4
```

- Concaténation : +

```
1 liste1 = [1, 2]
2 liste2 = [3, 4]
3 liste3 = liste1 + liste2
4 print(liste3)  # Output: [1, 2, 3, 4]
```

- Répétition : *

```
1 liste4 = [1,2] * 3
2 print(liste4)  # Output: [1, 2, 1, 2, 1, 2]
```

Opérations sur les listes

- Obtenir une sous-liste :

```
sous_liste = ma_liste[1:3]
print(sous_liste)  # Output: [3, 5]
```

- Tranches avec pas :

```
tranche = ma_liste[::2]
print(tranche)  # Output: [1, 5]
```

- Minimum, maximum, somme :

```
ma_liste = [-3,-2,-1,0,1,2,3]
print(min(ma_liste))  # Output: -3
print(max(ma_liste))  # Output: 3
print(sum(ma_liste)) # Output: 0
```

- Test si un élément est présent ou non dans la liste :

```
savane = ["lion","girafe","éléphant"]
"lion" in  savane # Output: True
"penguin" in savane  # Output: False
"ours" not in savane # Output: True
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Une chaîne est une séquence de caractères entre guillemets simples ou doubles. Une liste est une séquence d'éléments, où chaque élément peut être n'importe quoi (un entier, un flottant, une chaîne, etc.).

Une liste de caractères et une chaîne de caractères, ça n'est donc pas la même chose !

Les chaînes sont **"immuables"**, ce qui signifie qu'on ne peut **pas modifier** individuellement des caractères dans une chaîne.

Par exemple, ceci ne fonctionne pas :

Code (Erreur out of range)

```
1 >>> nom =  "jeff"
2 >>> nom[0] = "J"
3 Traceback (most recent call last):
4   File "<stdin>", line 1, in <module>
5 TypeError: 'str' object does not support item assignment
```

Relation entre liste et chaine

On peut facilement passer d'une liste à une chaîne et inversement.

- D'une chaine à une liste, on peut simplement *caster* avec `list()`

Code

```
1 chaine =  "bonjour le monde"
2 liste = list(chaine)
3 print(liste) #['b','o','n','j','o','u','r',' ','l','e',' ','m','o','
      n','d','e']
```

- D'une liste à une chaine, on peut utiliser la fonction `join()`

Code

```
1 liste =  ['c', 'o', 'm', 'm', 'e', 'n', 't', ' ', 'ç', 'a', ' ', 'v'
      , 'a']
2 chaine = ''.join(liste) #join tous les mots de la liste
3 print(chaine) #"comment ça va"
4
5 liste =  ["très", "bien", "merci"]
6 chaine = ' '.join(liste) #un espace entre chaque mot
7 print(chaine) #"très bien merci"
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Définition

Un ensemble est une **collection non ordonnée d'objets immuables*** et **uniques.** En d'autres termes, c'est une **structure de données** non indexée permettant de **stocker** des objets non modifiables sur place.

Propriétés des ensembles :

- Ils partagent les mêmes caractéristiques que les ensembles en mathématiques, **c'est la même chose.**
- On peut effectuer des unions, intersections, tester si un élément appartient à cet ensemble ou non...
- Chaque élément est présent une seule et unique fois, et ne peut pas être modifié
- On peut cependant ajouter et retirer des objets à tout moment

* Un objet immuable est un objet qui ne change pas, que l'on ne peut pas modifier (nombre entiers, chaîne de caractères...). À l'opposé, on a les objets mutables, comme les listes, les dictionnaires ou encore les sets.

Déclarer et initialiser un ensemble

On peut déclarer un ensemble vide :

Syntaxe (Déclaration d'un ensemble vide)

```
ensemble = set()
```

Ou avec des valeurs directement :

Syntaxe (Déclaration d'un ensemble avec des éléments)

```
ensemble = {1, 2, 3, 4}
ensemble = {"girafe", "tigre", "singe", "souris"}
ensemble = {1, "girafe", "a", 3.14}
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Opérations sur les ensembles

- Ajouter un élément `add(objet)` :

```
animaux = {"cheval", "mouton", "poulet"}
animaux.add("vache")
print(animaux) #{"vache", "cheval", "mouton", "poulet"}
```

- Mettre à jour un ensemble avec un itérable `update(iterable)` :

```
animaux = {"cheval", "mouton", "poulet"}
animaux.update({"vache", "cochon"})
print(animaux) #{"vache", "cochon", "cheval", "mouton", "poulet"}
```

- Supprimer des éléments :

```
animaux = {"vache", "cochon", "cheval", "mouton", "poulet"}

animaux.discard("poulet") #{"vache", "cochon", "cheval", "mouton"}
animaux.remove("mouton")  #{"vache", "cochon", "cheval"}

animaux.discard("bidule") #{"vache", "cochon", "cheval"}
animaux.remove("bidule")  #KeyError : "bidule"

animaux.clear() #set(), c'est vide !
```

Opérations sur les ensembles

- Union :

```
A = {1, 2, 3, 4, 5}
B = {6, 7, 8, 9, 10}

print(A | B)         # {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
print(A.union(B))    # {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

- Intersection :

```
A = {3, 4, 5, 6, 7}
B = {6, 7, 8, 9, 10}

print(A & B)                  # {6, 7}
print(A.intersection(B))      # {6, 7}
```

- Difference :

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print(A - B)                  # {1, 2, 3}
print(A.difference(B))        # {1, 2, 3}
```

Relation ensemble et liste

On peut passer d'une liste à un ensemble, c'est pratique pour éliminer les doublons par exemples !

Code (Eliminer des doublons)

```
1    liste = [8, 3, 3, 1, 5, 5, 1, 3, 8]
2    ensemble = set(liste)
3    print(ensemble) # {1,3,5,8}
4    liste_retour = list(ensemble)
5    print(liste_retour) # [1,3,5,8]
```

Attention toutefois à ce que les éléments de la liste soient hashables !

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Un tuple ressemble en de nombreux points à une liste, mais il est **impossible de modifier les données d'un tuple une fois défini.**

Définition

Un tuple est une **collection ordonnées d'objets** qui ne peut pas être modifiée une fois déclarée. On peut accéder à ses éléments via un indice.

Propriétés des tuples :

- Comme pour les listes, tout type d'objets.
- Il n'est pas possible d'ajouter / supprimer des données (comme on le faisait avec `l.append()` ou `l.remove()` pour les listes par exemple).
- Il n'est pas non plus possible de modifier un élément d'un tuple (si a est un tuple, on ne peut donc pas écrire `a[0] = 5` comme on l'aurait fait pour une liste).
- Ils sont souvent utilisés afin de retourner plusieurs valeurs dans une fonction.

Déclarer et initialiser un tuple

Quelques exemples d'utilisation :

Syntaxe (Déclaration d'un tuple)

```
t = (1,8,6)
t = (2.8, "lion", 6)
```

Les tuples sont très utiles pour stocker le résultat d'une fonction.

Code

```
1  def test():
2      return 3, 4, 5
3
4  res = test()
5  print(res) # (3, 4, 5)
6  print(res[1]) # 4
7
8  a, b, c = test()
9  print(a) # 3
10 print(b) # 4
11 print(c) # 5
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Définition

Un dictionnaire est une association non ordonnée de clés (keys) uniques immuables à des valeurs (values) mutables. En d'autres termes, c'est une **structure de données** dont les éléments sont indexés par des clés que nous choisissons nous mêmes.

Propriétés des dictionnaires :

- Les clés doivent être uniques et immuables (chaînes de caractères, nombres, tuples contenant des immuables)...
- On peut modifier le contenu des valeurs indexées par des clés, mais on ne peut pas modifier la clé.
- On peut à tout moment ajouter ou supprimer une clé et sa valeur d'un dictionnaire.

Déclarer et initialiser un dictionnaire

On peut déclarer un dictionnaire vide :

Syntaxe (Déclaration d'un dictionnaire vide)

```
dictionnaire = {}
dictionnaire = dict()
```

Ou avec sa combinaison clé-valeur (key :value) directement :

Syntaxe (Déclaration d'un dictionnaire avec des éléments)

```
dictionnaire = { key1 : value1 , key2 : value2 , ...}
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

Opérations sur les dictionnaires

- Ajouter ou modifier une paire *clé : valeur* :

```
etudiant = {"prenom":"Bob", "nom":"Delarochelle"}
etudiant["prenom"] = "Alice" #modification
etudiant["classe"] = "FISA3" #ajout
print(etudiant) #{'prenom': 'Alice', 'nom': 'Delarochelle', 'classe
    ': 'FISA3'}
```

- Accéder à un élément :

```
print(etudiant['prenom']) #Alice
print(etudiant['adresse']) #KeyError, car la clé n'éxiste pas
print(etudiant.get('adresse')) #None, évite l'erreur
```

- Supprimer des éléments :

```
del etudiant['classe']
etudiant.pop('classe') #recupere et supprime la valeur
etudiant.clear() #efface tout le contenu du dictionnaire
```

- Vérifier la présence d'une clé dans un dictionnaire :

```
print('prenom' in etudiant) # True
print('adresse' in etudiant) # False
```

Itérer sur un dictionnaire

```
etudiant = {'prenom': 'Alice',
            'nom': 'Delarochelle',
            'classe': 'FISA3'}
```

- Boucle for classique, marche uniquement sur les clés :

```
for key in etudiant:
    print(key)
```

- Pour boucler sur les clés, dict.keys() :

```
for key in etudiant.keys():
    print(key)
```

- Pour boucler sur les valeurs, dict.values() :

```
for value in etudiant.values():
    print(value)
```

- Pour boucler sur les clés et les valeurs en même temps, dict.items() :

```
for key, value in etudiant.items():
    print(key, value)
```

1 Préambule

2 Les Listes
- Définitions et caractéristiques
- Opération sur les listes
- Cas particulier : les chaines de caractères

3 Les ensembles
- Définitions et caractéristiques
- Opérations sur les ensembles

4 Les tuples

5 Les dictionnaires
- Définitions et caractéristiques
- Opérations sur les dictionnaires

6 La copie de structures

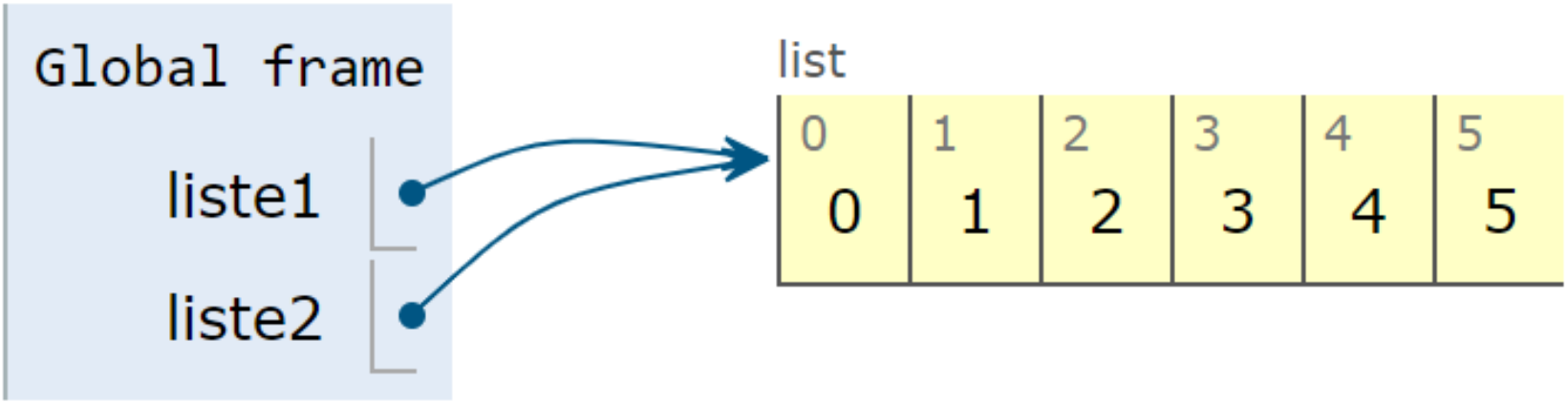
La copie d'une structure (mauvaise façon)

On peut parfois avoir besoin de **copier une structure**, par exemple une liste et son contenu, notamment pour en sauvegarder ou archiver l'état.

Il est alors **tentant** d'effectuer : `liste2 = liste1`, par exemple :

```
1 liste1 = [0,1,2,3,4,5]
2 liste2 = liste1
```

Mais cette instruction n'effectue pas une copie de l'objet, elle ajoute une nouvelle référence à l'objet :

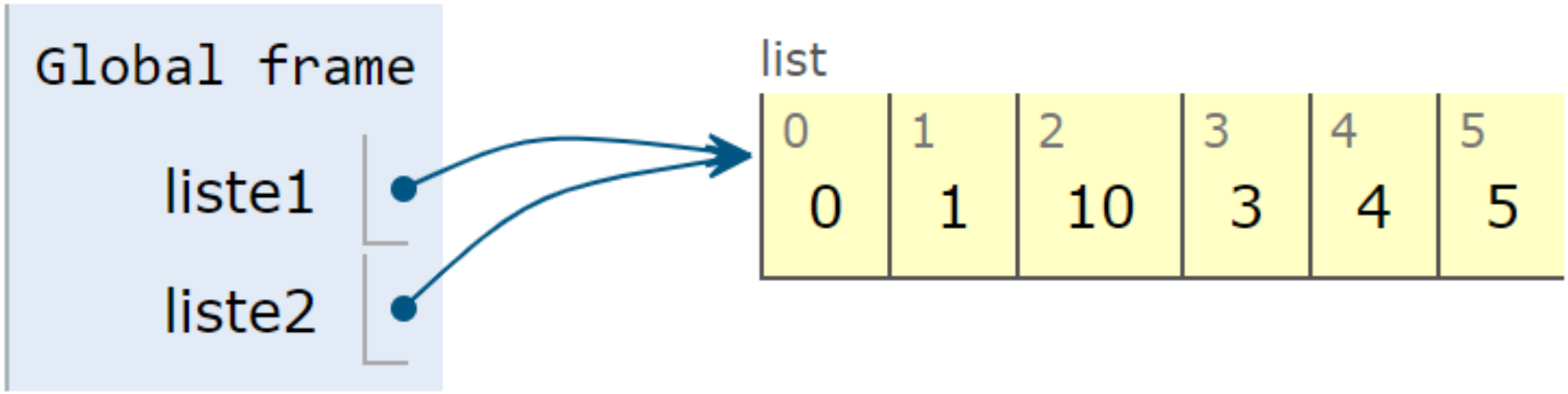


La copie d'une structure (mauvaise façon, conséquences)

En conséquence, toute modification de l'objet "pointé" par le nom `liste1`, sont les mêmes que pour le nom `liste2`, car c'est le même objet !

Ainsi, si l'on continue avec notre exemple :

```
1 liste1 = [0,1,2,3,4,5]
2 liste2 = liste1
3 liste2[2] = 10
```



On voit bien que la liste a été modifiée pour `liste1` et `liste2`. Nous n'avons finalement rien copié du tout.

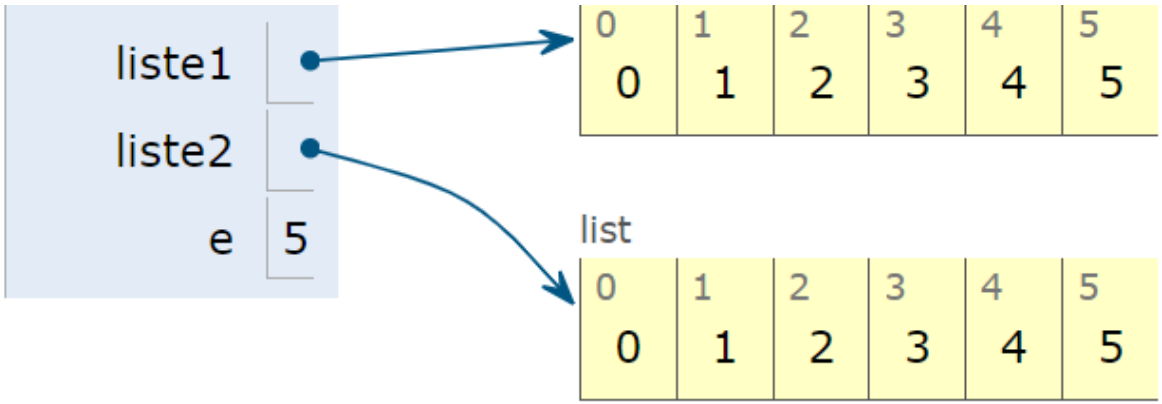
La copie d'une structure (plusieurs bonnes façons)

Pour être sur que l'on effectue une réelle copie d'une liste, il faudrait :

1. Créer une nouvelle liste vide
2. Parcourir un à un les éléments de la première liste
3. Ajouter un à un ces éléments à la deuxième liste

En code, ça donne :

```
1 liste1 = [0,1,2,3,4,5]
2 liste2 =  []
3 for e in liste1 :
4     liste2.append(e)
```



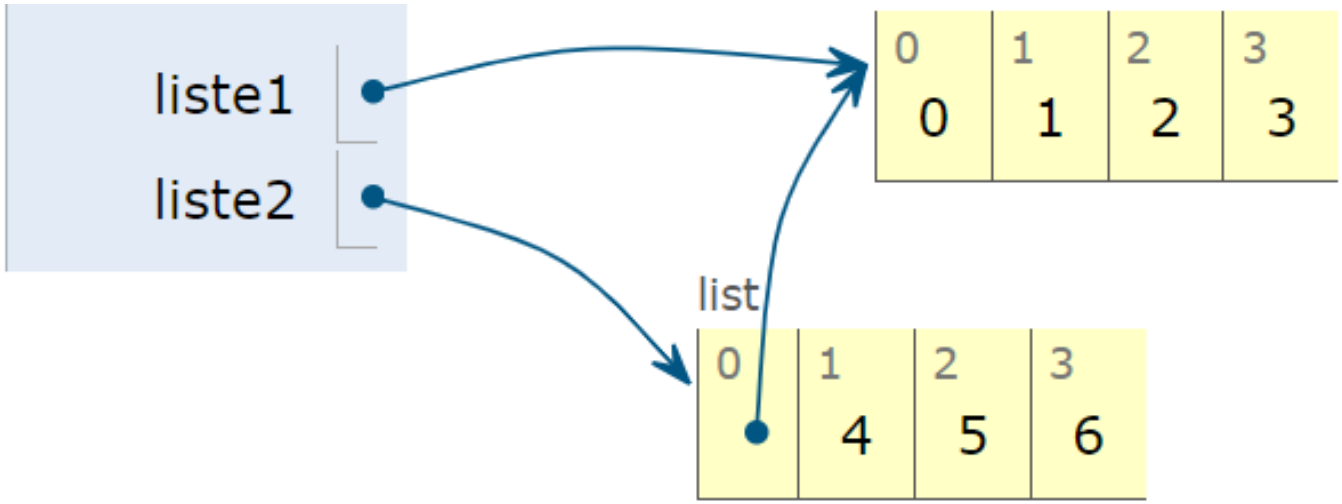
La copie d'un objet (plusieurs bonnes façons)

Heureusement, `Python` nous fournit des fonctions permettant de copier un objet simplement, il faut alors se poser une question : **l'objet à copier est il un objet composé ?**

Définition

Un **objet composé** est un objet **contenant d'autres objets**, comme des listes ou des instances de classe.

```
1 liste1 = [0, 1, 2, 3]
2 liste2 = [liste1, 4, 5, 6]
```

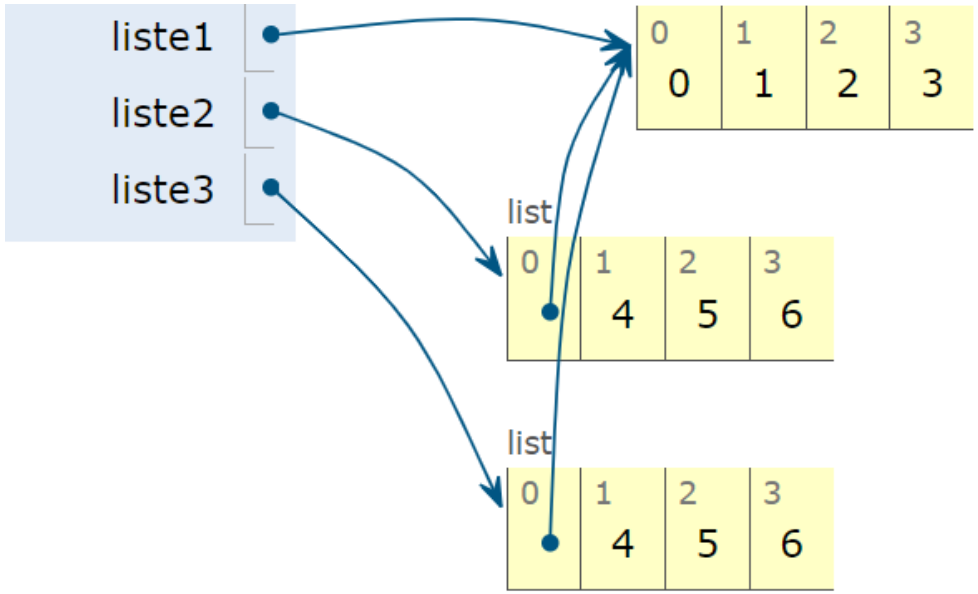


La copie superficielle

Définition (La copie superficielle)

Une copie superficielle **construit un nouvel objet** puis insère dans l'objet des **références** aux objets trouvés dans l'original.

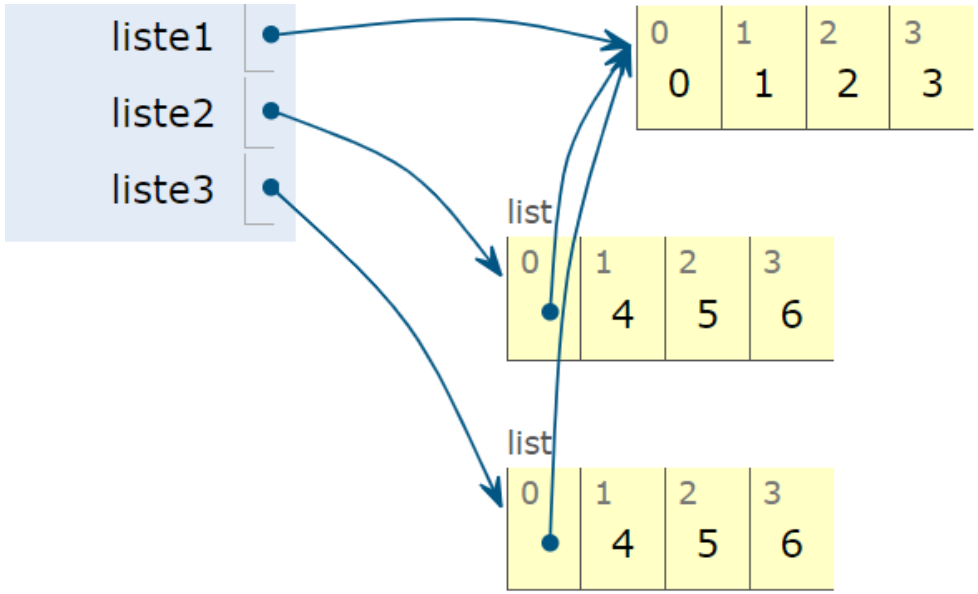
```
1 from copy import copy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = copy(liste2)
```



La copie superficielle : modification de l'objet original

Que se passe t'il si on modifie la liste 1 ?

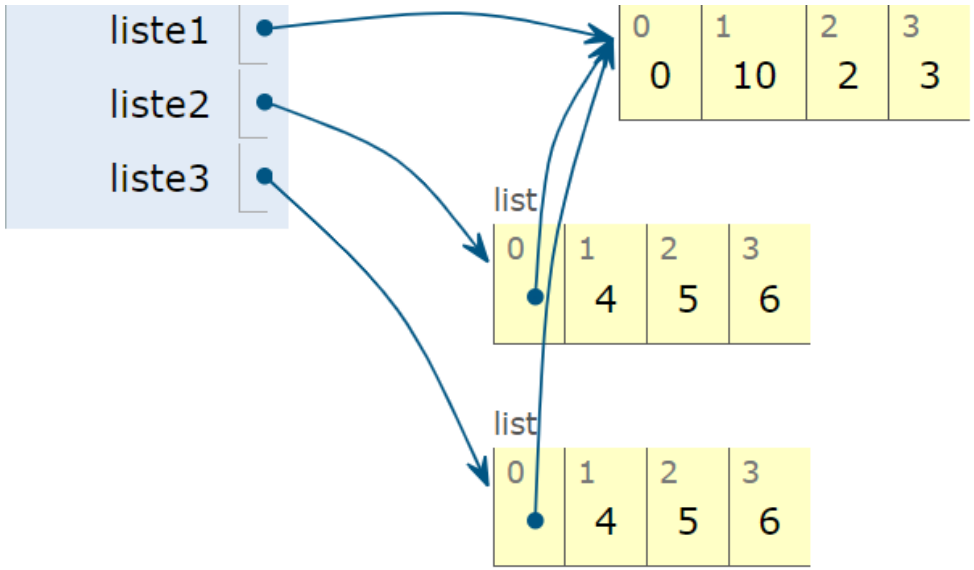
```
1 from copy import copy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = copy(liste2)
```



La copie superficielle : modification de l'objet original

Que se passe t'il si on modifie la liste 1 ?

```
1 from copy import copy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = copy(liste2)
5 liste1[1] = 10
```



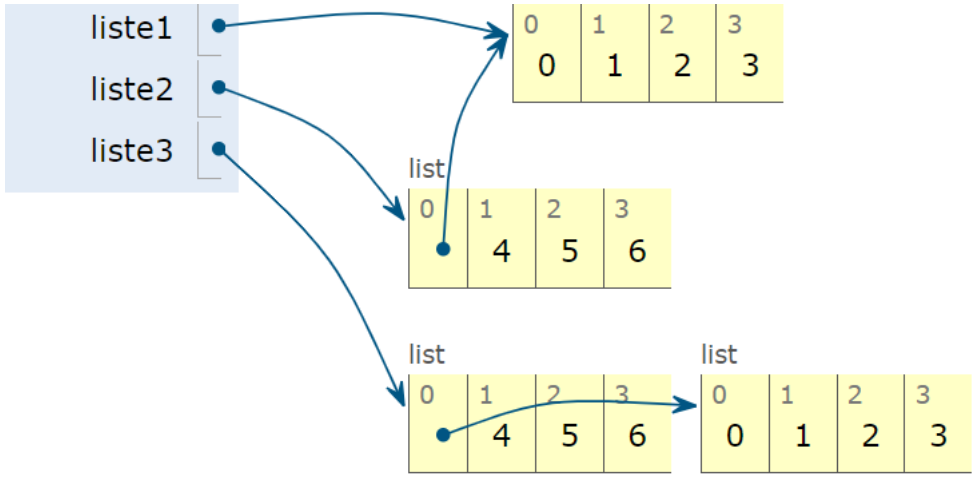
Le contenu des liste2 et liste3 est modifié aussi, car il s'agit du même objet liste1, il n'a pas été copié.

La copie profonde

Définition

Une copie profonde (ou récursive) **construit un nouvel objet** composé puis, **récursivement*, insère** dans l'objet composé **des copies des objets** trouvés dans l'objet original.

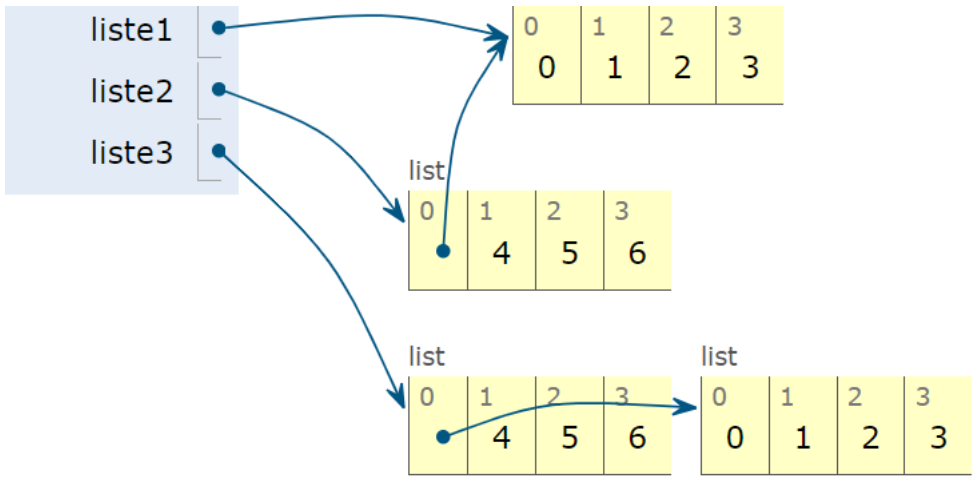
```
1 from copy import deepcopy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = deepcopy(liste2)
```



La copie profonde : modification de l'objet original

Que se passe t'il si on modifie la liste 1 ?

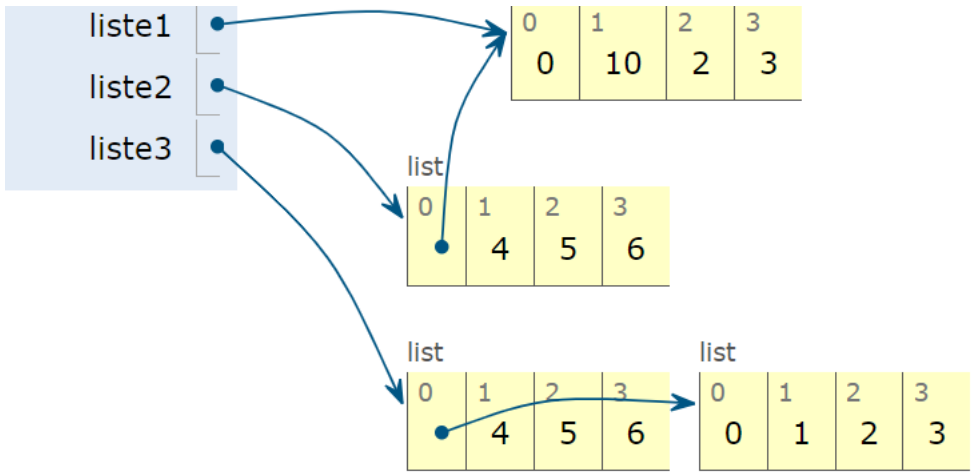
```
1 from copy import deepcopy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = deepcopy(liste2)
```



La copie profonde : modification de l'objet original

Que se passe t'il si on modifie la liste 1 ?

```
1 from copy import deepcopy
2 liste1 = [0,1,2,3]
3 liste2 = [liste1,4,5,6]
4 liste3 = deepcopy(liste2)
5 liste1[1] = 10
```



Le contenu de la liste3 n'est pas modifié, car la liste 1 copiée avec la copie profonde a bien généré une nouvelle liste.

Ce qu'il faut retenir de la copie

- Essentiel de bien comprendre son fonctionnement, très utile et important dans tous les langages informatique
- Une allocation = ne suffit par pour effectuer une copie
- La différence entre copie superficielle et récursive n'est pertinente que pour les objets composés
- package `copy`, avec `copy.copy()` et `copy.deepcopy()`
- `copy` : copie superficielle, elle copie les éléments simple et fait des références aux objets composés
- `deepcopy` : copie profonde, elle copie tous les objets qu'ils soient simples ou composés